

Neural Style Transfer

Sungwon Kim
skim434@jhu.edu

TaiMing Lu
tlu37@jhu.edu

Yiyang Li
yli302@jhu.edu

Chengze Nao
cnao1@jhu.edu

Chenxuan Xiang
cxiang4@jhu.edu

I. INTRODUCTION

Neural image generation, including neural style transfer, is gaining popularity and finding practical applications in industries like entertainment, advertising, and art. Neural style transfer has been used widely in the industry such as image and video editing, AR and designing, as well as in our daily lives with the example of selfies changed into cartoon style, or meme pictures reconstructed with cool artistic styles. With the prevalence of this content, we were compelled to explore and understand this technology, and if possible, improve it to our needs.

There are many prior works in this field, including CNN image style transfer [1], CycleGAN [2] and Diffusion models [3]. All of the models had good results despite using different approaches. CNN image style transfer utilized the fact that the CNN model would automatically extract different levels of features in its respective layers, and with that it is able to combine different levels of features from the style image with the content image for the final generated image. CycleGAN used the GAN approach, with two pairs of generators and adversaries with the focus to distinguish the back-and-forth transformation between the two image domains. Lastly, a Diffusion-based model [3] used semantic latent space called h-space from checkpoints of pre-trained diffusion models for style transfer without extra training.

All the models mentioned above had great potential, yet due to time limitations, we decided to spend most of the time examining the CNN image style transfer. Our goal here is to fully explore the model, understand how it works, what are its results and what each parameter does to the final results. We modified the model by testing out different parameters, changing the underlying CNN model, adding more losses and modifying the transfer pipeline. Through these modifications, we were able to change the emphasis on different aspects of style transferring, as well as solve some common style transferring problems including color transferring, a phenomenon where the color of the style image dominated in the final generated image.

II. METHODS

A. Data

Both content images and style images are from Kaggle Best Artworks of All Time. dataset [4]. It includes paintings of the 50 most influential artists of all time. Images differ in dimension, many of them have dimensions around (500, 400, 3). For this research report, we decided to focus on two images. *The Abduction of Europa* by Rembrandt and *The Trinquetaille Bridge* by Vincent Van Gogh.

B. Algorithm

We used CNN based Algorithm introduced in the Neural Algorithm of Artistic Style by Gatys et. al [5].

The algorithm requires a pre-trained CNN. Researchers used VGG19 as the pre-trained CNN in the paper. The algorithm applies the same CNN to the content image, style image and generated image. it uses intermediate convolution layer outputs to compute loss.

Normally, neural networks train the model weights. However, the paper algorithm trains the pixels of the generated image. The generated image starts from random noise and is backpropagated to minimize loss. The loss function is $L = \alpha L_{\text{content}} + \beta L_{\text{style}}$.

For content loss L_{content} , we first acquire intermediate output of the deep convolution layer (block5_conv2) of the content image and the generated image. The square difference between the two intermediate outputs as L_c .

For style loss L_{style} , we acquire multiple intermediate outputs of convolution layers of style image and generated image. Gram matrix is calculated for each layer. Then we sum up square differences between the style gram matrix and generated gram matrix to get content loss.

The final loss is the weighted sum of content loss and style loss. We test out α and β ratio between 10^2 to 10^{10} .

C. Improvement

Below are the additional changes we made to the original algorithm.

- We changed the initialization of the generated image.
- We added additional loss, L_d . The Loss function is now $L = \alpha L_c + \beta L_s + \gamma L_d$. The loss metric for L_d include
 - MSE between the content image and generated image
 - Variation loss of the generated image
 - Edge difference between content image and generated image.
- We also try the “reverse pipeline” architecture. One of which is submitting the generated image as a content image, the original content image as the style image to generate a new image. This architecture seems to retrieve data lost from the original content image.
- We change the underlying neural network structure from VGG to ResNet. This would change the layers and the corresponding traits chosen, and we can explore how these would influence the final style transfer.

We have submitted ipynb file for image generation.

D. Evaluation

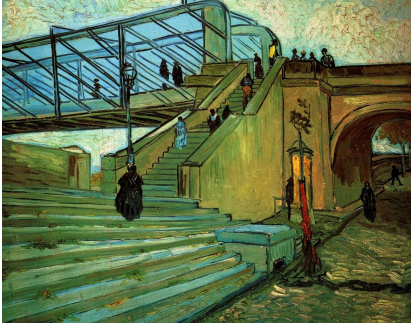
Since "style" is a subjective concept, it is hard to come up with an exact numerical evaluation of different results and simply say one is better than the other. Our result and discussion assessments of the outputs would be based on describing the features of each of the outputs, with a focus on where the style transfer didn't work well, including overly-noisy images, color transferring(see IV.C), or images without substantial changes when compared to the original content image. As for images with substantial changes from the original content image, we would not compare them side by side since each person could find beauty in different outcomes. Our results and purposes are involved in understanding how to avoid failures when using the model, and presenting possible modifications so that readers could understand and choose methods based on individual interests.

III. RESULTS

The images used in this project for comparing results are:



(a) Content Image

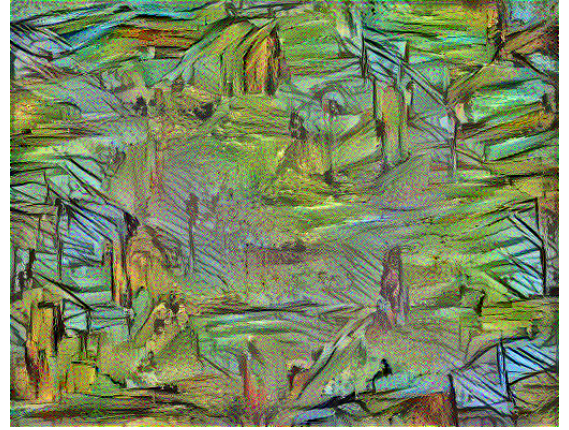


(b) Style Image

Fig. 1: Content and Style Image



(a) Random Noise with same initial loss

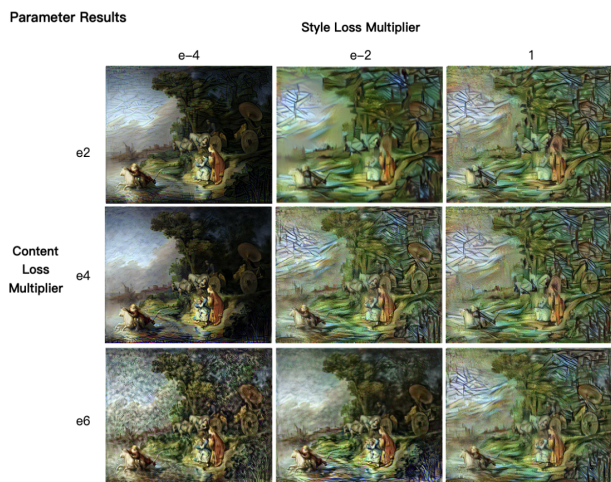


(b) Random Noise with suggested weights

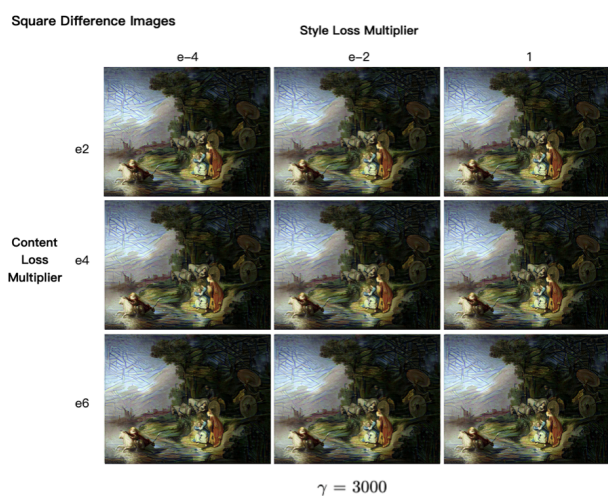
Fig. 2: Random Noise Initialized image

And with the paper's suggestions, results from randomly initialized weights are shown in Fig.2. These images show some of the problems from the original paper: random noise initialized results could look noisy too, as seen in Fig.2a. In some worse cases we could get failures similar to Fig.2b, where barely any features from the content image are kept.

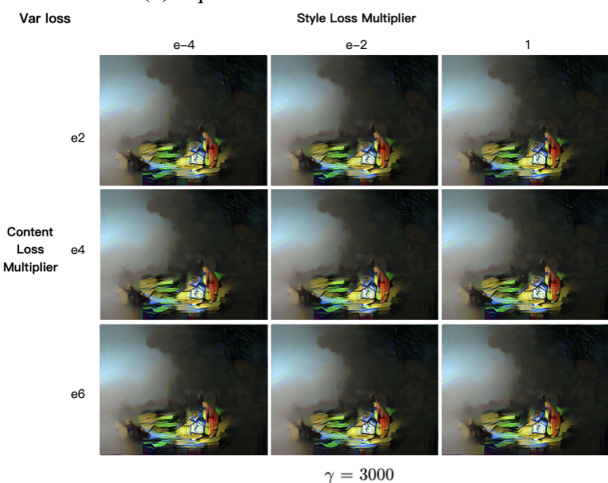
Due to this, we switched our initial generated image to content image. In general, switching to content image initialization allowed us to keep more features from the content image, and avoided the generation of completely random images similar to Fig.2b. Results are shown in this section, and for additional generated images, they could be found here.



(a) Vanlia Results



(b) Square Difference Added Loss



(c) Variation Loss

Fig. 3: Loss Result Matrices

Fig.3 display results for different parameter changes, where (a) is the normal content and style loss from the paper with

different values tested, (b) is the result of adding a new loss for square difference, and (c) is new loss added for variation. From (a) we could observe how loss multiplies influence the final output image, that with a higher style loss multiplier, we get more features from the style feature; and more content loss multiplier brings more content image features, which could counteract with some of the effects with high style loss multiplier.

For 3b and 3c, we could observe how newly added losses would influence the original model. Adding a square difference between generated image and the content image suppresses the style transfer. We can see in (b), even with different α, β weights, generated images do not have much difference. 3c displays images with added variation loss. the background is smoothed and the characters had more style changes from the style image. Overall, we could also observe that with more losses added, the original content and style loss had less effect on the resulting outputs.

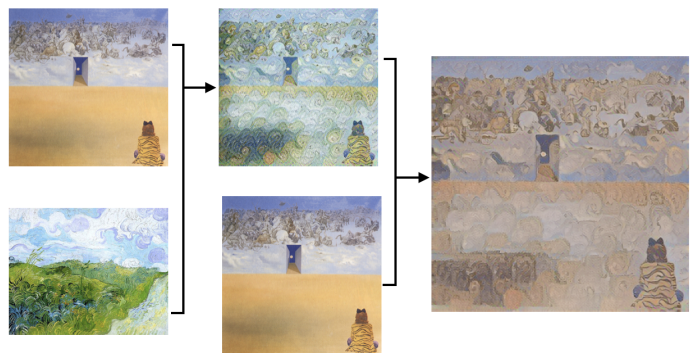


Fig. 4: Example Result from Changing Pipeline

Fig.4 are the results from pipeline changes. This one is quite intuitive: by rerunning the process with the original image, we were able to get a style-transferred image that also maintains colors from the original image.

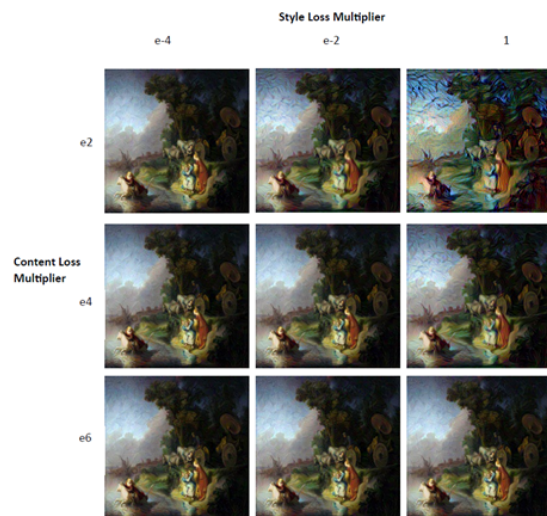


Fig. 5: Resnet50-based style transfer

And finally, Fig.5 shows the results when the underlying model is changed to Resnet50. We could observe substantial changes compared to Fig.3a, where Resnet50 is more reserved, and more aggressive parameters are needed for significant change in styles. We could only see obvious influences from the style image in the result on the top right, where the style loss multiplier $\beta = 1$ and content loss multiplier $\alpha = e2$. Resnet50's results also exhibit a different style compared to the normal VGG19 architecture, and with more parameter tuning, it could be a good substitute for VGG19 in style transferring.

IV. DISCUSSION

A. Random Initialization

The original implementation of style transfer set the initial generated image as noise. As seen in Figures 2, These initialization do not look great. Figure 2a looks like a noisy image of the content image while figure 2b doesn't have any content information. In particular, they are sensitive to loss ratio. Furthermore, these images take a long time to train. They also tend to be stuck at the local minimum.

Because of these problems, we decided to initialize the generated image as the content image. We see that this method provides better results.

B. loss curve

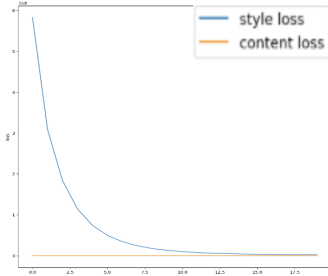


Fig. 6: Style Content Loss Graph

We plotted the loss of content and loss according to suggestions in the paper. We see that style loss is much greater than that of content loss. Due to this observation, we tested different loss ratios for image generation.

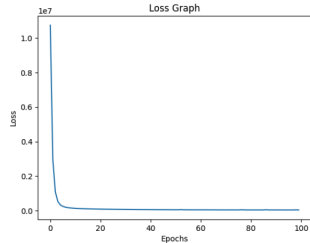


Fig. 7: Loss Graph

Figure 6 shows loss with respect to epochs after fixing the initial generated image to the content image. Each epoch contains 100 iterations of training. We see that the curve

plateau after 10 epochs. Thus, we trained our images on 1000 iterations. Training takes around 75 seconds per image on google colab.

C. Color Transfer and Solution

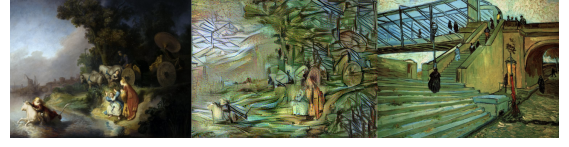


Fig. 8: Color Transfer

We observed that the color scheme of the feature image transferred to the style image. While this may be a desirable outcome in certain situations, we determined that it could be considered a bug in some cases. We hypothesized that incorporating the mean square error between the generated image and the content image would allow the generated image to adopt the color scheme of the content image. So we redefined the loss function as follows.

$$L = \alpha L_c + \beta L_s + \gamma L_d$$

Where L_d is a mean square error. As seen in figure 3b, and figure 9 the added loss successfully fixes the color scheme of the content image.

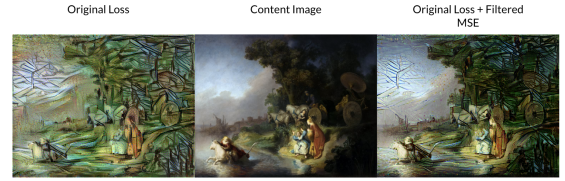


Fig. 9: Color Transfer Fix

We tested additional loss metrics such as edge difference, linear loss, and variation loss. We also implemented a smoothing filter pipeline which results in fewer outstanding pixels. The images can be found at this link. Method for calculating loss and γ values are present in the submitted code.

D. Changing underlying structure

We also changed the underlying backbone of the style transfer network to ResNet50. Since ResNet50 has more layers than VGG19, we hypothesize that it is more effective at capturing complex features compared to VGG19. Additionally, ResNet utilizes skip connections that allow information from earlier layers to be propagated forward to later layers, which can help prevent the vanishing gradient problem and improve the overall performance of the network. For the loss function, a similar convention is adopted, where the last block's output is used to calculate the content loss and for the style loss, outputs of all 4 basic blocks are used. Figure 5 is the result matrix using the same parameters as the VGG19-based style transfer network. When changing the model to extract the features, even with the same parameters, the result is not the same as

the VGG19-based style transfer neural network. However, it still has the same tendency to change the image as in the VGG19 model. A big content loss multiplier corresponds to less style imposed on the content image while a big style loss multiplier corresponds to a more styled image.

To solve the color transferring problem, we combined the ResNet50-based style transfer and a new loss term called grey loss. Grey loss has the same calculation format as style loss except that a greyscale version of the style image is used. Noted that original style loss is still used so there are three loss terms in total. Figure 10 shows the result of ResNet50-based style transfer with a content loss multiplier of 100 and a style loss multiplier of 1. Figure 11 shows the result of ResNet50-based style transfer with the same parameters as figure 10 with an additional grey loss term with multiplier 1, which is the same as the style loss multiplier. We can observe from the results that adding a grey loss term does prevent color transferring while maintaining style. Since the style loss multiplier and grey loss multiplier have the same value, style loss and grey loss converge with almost the same scale.

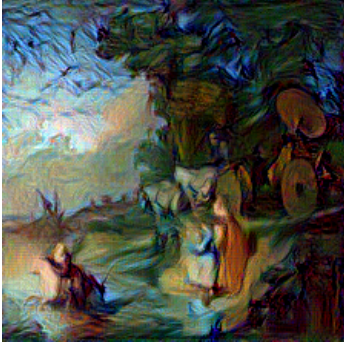


Fig. 10: Result image without grey loss

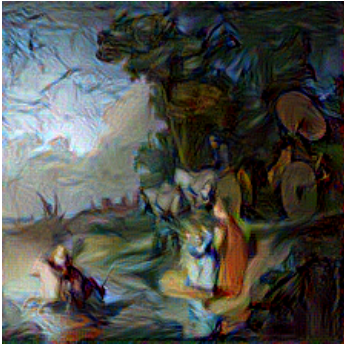


Fig. 11: Result image with grey loss

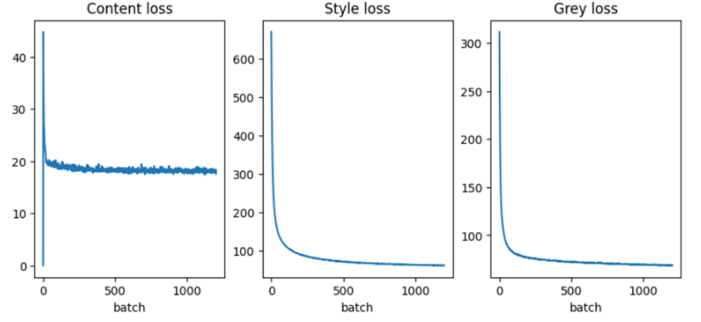


Fig. 12: Loss Graph

E. Changing the processing pipeline

As another method for solving the color transferring problem, we have designed a new pipeline for handling images. In this pipeline, we use the original model and save the output. In the next stage, the output image is used as the content, and the original content image is used as the style. We run the transfer network once again to restore the content image's color and style. As shown in Figure 4, this processing pipeline successfully maintains the color while smoothly combining the style. However, this pipeline has one shortcoming: the effect of the two separate neural style transfer stages is hard to balance. If the content and style images have a high difference in terms of color, geometry, edges, curves/lines, and more, either stage in the pipeline will significantly modify the original content and discard its original features completely. We can solve this problem by fine-tuning the parameters on a case-by-case basis.

F. Future work

Our current process of finding the γ value in the VGG19 base model is based on an "educated guess." We can initialize the γ value such that $\gamma L_d = 2(\alpha L_c + \beta L_s)$ for a generated image trained to minimize the $\alpha L_c + \beta L_s$ loss. We then adjust the γ value based on our needs after looking at the result.

The L_d loss curve can be finicky because, in most cases, all but one L_d value is minimized when the generated image is equivalent to the content image. In other words, most L_d values start from 0 during training. In future work, suggested γ values could be generated using math.

Furthermore, our current L_d values are correlated with the closeness of the generated image to the content image. In future work, a loss function that depends on the closeness to the style image could be devised.

Additionally, recent works have proposed multiple quantitative evaluation metrics. We can try to build a testing pipeline that provides quantitative results for style-transferred images.

V. CONCLUSION

Based on our observations, we have the conclusion that in order to achieve certain results we desire, the following can be applied:

- Modifying parameters would bring different models catered to different needs. As from the example in the discussion, increasing edge detection's weight would produce results with edges more similar to the original picture.
- Adding additional loss metrics to the total loss steers the generated image to exhibit certain qualities. For example, adding a grey loss term can reduce the color transferring problem. We were also able to smooth out or sharpen generated images by adding additional loss parameters.
- Changing the pipeline of the model could also achieve some specific results we desire. For example, by retraining the output with the original input we get to keep more color features from the original picture, showing that alternating the pipeline could also be a potential way to adjust the weight of content and style picture in the final output.
- By changing the underlying feature extraction neural network, we could achieve different style transfer effects, or increase the efficiency/effect of the overall algorithm.

In conclusion, experimenting with different parameters, additional loss metrics, modifying the backbone network, and changing the processing pipeline for neural style transfer has yielded interesting results. By tweaking the original pipeline, the neural style transfer network can be adapted to cater to different needs and generate images with specific qualities. These findings suggest that there is potential for further exploration and improvement in the field of neural style transfer, particularly in developing more sophisticated techniques to better blend style and content.

REFERENCES

- [1] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2414–2423.
- [2] X. Fu *et al.*, "Digital image art style transfer algorithm based on cyclegan," *Computational Intelligence and Neuroscience*, vol. 2022, 2022.
- [3] J. Jeong, M. Kwon, and Y. Uh, "Training-free style transfer emerges from h-space in diffusion models," *arXiv preprint arXiv:2303.15403*, 2023.
- [4] ICARO, "Best artworks of all time," <https://www.kaggle.com/datasets/ikarus777/best-artworks-of-all-time>.
- [5] L. A. Gatys, A. S. Ecker, and M. Bethge, "A neural algorithm of artistic style," *arXiv preprint arXiv:1508.06576*, 2015. [Online]. Available: <https://arxiv.org/abs/1508.06576>

APPENDIX

Contribution table:

Contribution Table			
Name	Coding	Presentation	Write up
Sungwon Kim	20%	20%	20%
TaiMing Lu	20%	20%	20%
Yiyang Li	20%	20%	20%
Chengze Nao	20%	20%	20%
Chenxuan Xi-ang	20%	20%	20%

Detailed contribution can be found on this link.